

Seed, Generate, Reduce, Repair, Gate: A Closed CIRCT Bug Loop

Abhinav Venkata Kota, Adithya Jillellamudi, Priyesh Shukla

International Institute of Information Technology, Hyderabad

{abhinav.kota, adithya.jillellamudi}@students.iiit.ac.in, priyesh.shukla@iiit.ac.in

Abstract—CIRCT is an open-source compiler that turns hardware descriptions into Verilog, the text that chip design tools consume, so a defect inside it can corrupt a design unnoticed. Agents that repair CIRCT take their work from its public issue tracker, a queue users fill and maintainers, whose attention is the scarce resource, must drain. We describe a loop that makes its own supply: an agent reads a past bug fix, states the class of mistake it corrected, and writes fresh inputs probing for the same mistake elsewhere in the compiler. A fixed harness then builds, judges, minimises, deduplicates and repairs, and a gate ending in human approval decides what, if anything, is filed. We evaluate the seeded agent against a mutation fuzzer at equal, pre-registered budget under identical tools. We release the loop as reusable nodes and tools for the CHIA agent framework, with its seed corpus, transcripts and verdicts.

I. INTRODUCTION

CIRCT is an open-source compiler for hardware. A designer writes how a circuit should behave, in FIRRTL (the form the Chisel hardware language produces) or in SystemVerilog, and CIRCT lowers it to Verilog, the text that synthesis and layout tools turn into a chip. It is built on MLIR, the LLVM project’s framework for assembling a compiler out of interchangeable intermediate languages, and it inherits LLVM’s way of testing: each regression test is a file carrying the exact command to run and the output pattern to expect, executed by a runner called `lit`.

A CIRCT bug that crashes the compiler is an inconvenience: the designer sees it and works around it. A bug that makes CIRCT emit wrong Verilog is not, because the output is well formed, passes the tools below it, and can reach fabrication unnoticed.

CHIA [1] is the agent framework this hackathon is built on. An agent here is a language model wired into a program that reads files, runs commands and decides what to do next. CHIA ships an issue solver for CIRCT that chains six phases: assess the issue, reproduce it with a script, fix it, verify by rebuilding and rerunning the whole test suite, repair any regression the fix caused, and write the pull request description. Run on 16 CIRCT issues it fixed five, and three of those became merged pull requests. It writes nothing to GitHub; both of its flows only read.

Its supply is the constraint. Over the past year CIRCT received 82 issues labelled as bugs and closed 65, so the project is neither short of reports nor ignoring them. What does not happen is the backlog draining: 101 bug-labelled issues are open, at a median age of 911 days. The scarce

resource is maintainer attention, not bug supply, and an agent that empties the queue faster creates none. What would help is a bug that arrives already minimised, already checked against what is known, already carrying a candidate fix, from a source that files nothing else.

Past fixes are unspent information. A commit that corrects one wrong assumption inside a compiler pass usually leaves the same assumption standing at other sites, and CIRCT ships no fuzzing infrastructure in its own tree to find them.

This paper describes a loop that manufactures its own supply and then withholds most of it. We contribute the loop design (Section II), two preconditions we measured and repaired before it could run (Section III), and a pre-registered head-to-head against a mutation fuzzer (Section IV).

II. THE LOOP

Fig. 1 names the stages and marks the ones the agent runs. The agent proposes inputs and writes prose; the tools decide everything else, so no number in the result comes from inside the model.

Read the seed (1). A seed is one of 187 CIRCT commits from the last two years that look like bug fixes, each with its diff and the test its author added or changed. The agent reads both, states in one sentence the root-cause class the fix corrected, and names other sites in the compiler where the same assumption still stands.

Write probing inputs (2). The seed’s own test decides the language of the new inputs. Its command line names the tool through which the bug was reachable: `circt-opt`, which runs a named sequence of compiler passes over an intermediate file; `firtool`, the end-to-end driver from FIRRTL to Verilog; or the SystemVerilog front end. About 127 of the 187 seeds sit in code that no FIRRTL input can reach, so a generator that emitted only FIRRTL would be blind to most of the corpus.

Build (3). CIRCT compiles against one pinned commit of LLVM and MLIR, and building those from source costs hours; Section III explains how the loop avoids that. The build also compiles in CIRCT’s assertions, the internal consistency checks its own authors wrote into the source, which stop the compiler the moment an assumption breaks instead of letting the damage travel into the output.

Judge (4). An oracle is the rule that decides whether an input has found a bug. The primary one admits no argument: the compiler crashed, or one of its assertions fired. A second oracle runs the same design through CIRCT’s simulator and

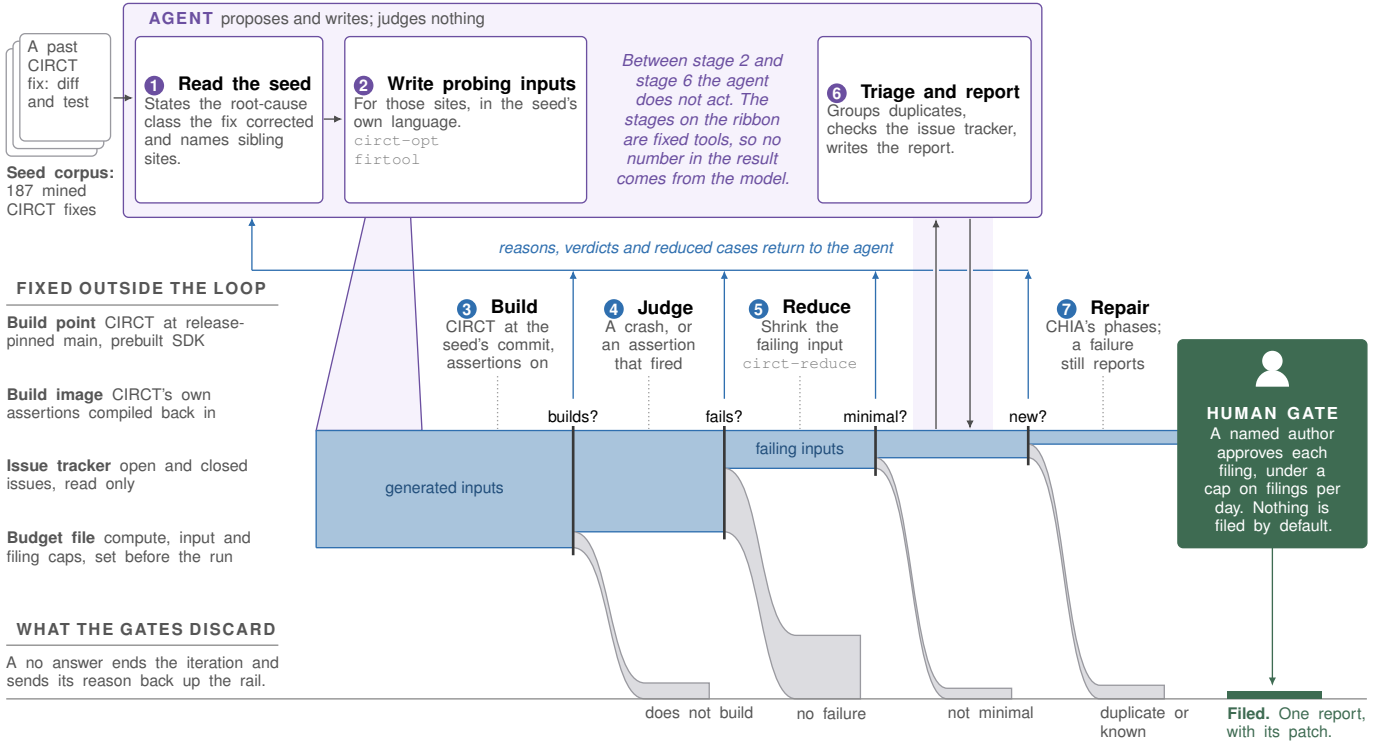


Fig. 1. Seeded CIRCT bug loop, a CHIA loop for turning past fixes into new, checked, repair-carrying bug reports. The agent holds the violet band at the top and acts three times only, at stages 1, 2 and 6; the blue ribbon below it is fixed apparatus, so no number in the result comes from the model. The ribbon is the candidates in flight. It is poured wide out of stage 2 and pinched by a bar at each gate, a question with two answers: the share answering no peels away onto the floor and ends its iteration, the rest carries on, and the reason climbs the thin rail back to the agent. Only a sliver survives to the human gate, which files nothing without a named author's approval. Ribbon and peel widths are illustrative, not measured.

through Verilator, an independent and long-established one, and treats disagreement as suspicious. It produces reports only, because the two disagree routinely over uninitialised and reset signals.

Reduce (5). An input that triggers a crash is usually large and mostly irrelevant to it. Reduction is the mechanical search for a smaller input that still fails: `circt-reduce`, which CIRCT ships, deletes parts of the input and keeps each deletion while the failure survives. A maintainer then reads the case in a minute, not an hour.

Triage and report (6). Many failures are one failure. Deduplication decides which candidates are the same bug, by assertion text and source location, by symbolised stack frames, and by a structural hash of the reduced input. Survivors are checked against CIRCT's open and closed issues and the commits made since the build point. The agent classifies each survivor as a bug, an invalid input or a known issue, and writes the report a maintainer would read.

Repair (7). CHIA's phase chain runs unaltered; the only addition is an entry point that accepts a local report in place of a GitHub issue. Repair is restricted to crashes and assertion failures, where nobody has to argue about what the right answer was.

The gates. Between stages sit questions with two answers, drawn as the bars that pinch the ribbon in Fig. 1: does it build, does it fail, is it minimal, is it new. A no ends the iteration,

skips every stage downstream of it, and returns the reason to the agent with the oracle verdicts and reduced cases. The last gate is a person. A named author approves every filing, under a cap on filings per day, and the loop files nothing by default.

III. TWO MEASURED PRECONDITIONS

The loop has to build CIRCT at arbitrary points of its history, and the compiler has to be able to complain. Neither held when we started. Table I lists what we measured, and this section gives the conclusions.

A. Building the past cheaply

CIRCT records the exact LLVM commit it expects, called the pin, and a build against any other fails outright rather than degrading, stopping in the first few targets. Compiling that LLVM from source costs hours per seed and would consume the campaign's budget. The project also publishes release tarballs of prebuilt LLVM and MLIR libraries, an SDK, each recording the pin it was built against. Where a release shares a seed's pin, CIRCT rebuilds in minutes. We mined 187 fix-shaped commits from 24 months of history and found an exactly matching SDK for 171 of them. To check that the mechanism works rather than merely counts, we rebuilt one seed from a stock release: its own test failed at the commit before the fix and passed at the fix.

The same mechanism fixes where the loop runs. It does not run at the newest commit of CIRCT's main branch, whose pin

TABLE I
WHAT WE MEASURED, AND WHAT IT MEANS

Measurement	Result	What it means
<i>Building CIRCT at a past commit</i>		
Fix-shaped commits mined from 24 months	187	The seed corpus: each is a past bug with its fix and its test.
Seeds with an exactly matching prebuilt SDK	171	Nearly all of them rebuild in minutes rather than hours.
One full build, assertions off, <code>circt-opt</code> targets only	1,027 targets, 939 s	Cheap enough to redo per seed. The seed’s test was red before its fix and green after.
Pin windows with no matching release	16 of 63	Sometimes no buildable point exists. The worst run lasted 100 commits and 16 days.
Lag from a pin change to a usable release	3.3 d median, 33.5 d worst	The loop’s build point trails the branch head by days.
<i>Letting the compiler complain</i>		
Compile commands defining <code>NDEBUG</code>	620 of 620	Assertions are absent from the stock CHIA image.
Assertion-handler references in the built binary	0	The primary oracle would have nothing to detect.
Targets that build with <code>-UNDEBUG</code>	605 of 605	Restoring the assertions does not break the build.
Object files carrying the handler again	495 of 513	The checks are back across the compiler.
Core dialect and conversion tests passing with assertions on	520 of 523	CIRCT still behaves. The three failures trace to a stale SDK tool.

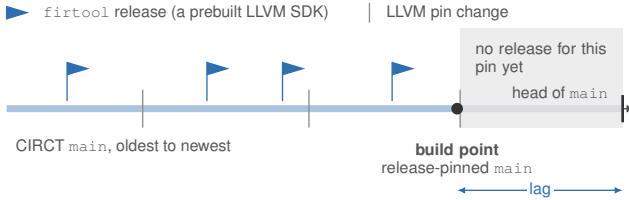


Fig. 2. Release-pinned main. Changes to the LLVM commit CIRCT pins cut its branch into windows, and a prebuilt SDK exists for a window only where a release falls inside it. The loop builds at the newest commit whose pin has a release, which is the head of the branch only while the current window has one. A bug fixed inside the lag is found again by the loop and dropped as already known.

has no published release today. It runs at release-pinned main, the newest commit whose pin does have one. That point trails the branch head by days, as Fig. 2 shows.

B. Letting the compiler complain

Optimised builds delete assertions by defining the macro `NDEBUG`. CHIA’s CIRCT image is such a build: every one of its 620 compile commands defines that macro, and the resulting binary holds no reference to the assertion handler at all. A loop whose primary oracle is a failed assertion would find nothing under the stock image, whatever the agent wrote. One compiler flag, `-UNDEBUG`, puts the assertions back. The build still completes, nearly every object file carries the handler again, and the core dialect and conversion tests still pass on all but three, which fail for an unrelated reason (a stale tool in the SDK). The restored binary still describes itself as an optimised build, so every report the loop files has to name the flag.

One caution about Table I: the timed build was made with assertions off and covered only one of CIRCT’s tools, so it is a lower bound on what the loop’s image will cost.

IV. EVALUATION DESIGN

The experiment asks whether reading a past fix makes an agent better at finding new bugs than mutating the same tests at random. Two arms run the same harness, oracles, reducer and gate; only the generator differs. The seeded arm is stages 1 and 2 above. The mutation arm starts from the same seed tests and applies syntactic mutations without ever seeing the diff or the root-cause class, following Mut4All’s design for mutators synthesised by a language model [2].

Both arms spend the same pre-registered budget: the compute allowance and the caps on generated inputs and on filings are committed to the public loop repository before the campaign starts, so no threshold can be chosen afterwards. The headline result is the number of distinct bugs a CIRCT maintainer confirmed, by labelling, commenting on or fixing them, within that budget. Our own oracle agreeing with itself produces a candidate, not a bug. The secondary counts are candidates before and after deduplication, reports filed, the fraction of filed reports maintainers accepted, and repairs merged.

If both arms find nothing, the paper reports that. Pre-registration is what makes such an outcome publishable, and it carries information: CIRCT’s assertion surface would not be reachable at this budget by either strategy.

Contamination. CIRCT’s history is public and sits in the training data of any model we can use, so the agent may reproduce a fix it memorised. Every candidate is screened against fixes made after its seed, and a match is recorded and reported apart from the rest. A match does not make the candidate invalid, since a real crash is a real crash, but it changes what the number means. The repair stage runs CHIA’s executor unchanged and is not screened.

Maintainer protocol. We post the method to the project’s forum before the campaign starts, and leave issues marked as good first issues to human newcomers.

Failure taxonomy. Every candidate reaching the gate is recorded with its verdict, one of unreproducible, not minimal, invalid input, duplicate, undecided or new bug, and every repair attempt with the phase that failed. The record shows where the loop spent budget without return.

V. RELATED WORK

Agentic repair. SWE-bench [3] scores agents on real GitHub issues and set the pattern this work inherits; CHIA’s issue solver [1] applies it to a hardware compiler. Both draw from a queue; we add the stage that fills one.

Compiler fuzzing. CSmith [4] generates random C programs and found previously unknown bugs throughout production C compilers. For MLIR, the framework CIRCT is built on, MLIRod mutates along operation dependencies [5], Nüwa adds generation driven by a language model [6], DESIL targets silent wrong-output bugs [7], and FLEX interleaves learning with exploration and found 80 previously unknown bugs in a

30-day campaign [8]. None of them repairs or withholds; they hand a human a pile of crashes. Mut4All synthesises mutators from past bug reports [2], and our mutation arm follows its design.

Seed-guided generation. AFuzz [9] is the origin of the method: given a reference bug, an agent analyses the root cause, hypothesises where else that cause could live, and tests each hypothesis with generated code, finding 40 bugs in V8 in about a month and 19 more in two other JavaScript engines. We apply it to a hardware compiler, behind a repair chain and a gate.

Reduction. Delta debugging [10] is the algorithm the reducer in stage 5 implements. We use the tool as supplied and contribute only its placement inside the loop.

Gating and abstention. Abstain-and-Validate raises repair success by letting a second model decline a bug and reject a patch [11]; llvm-harness packages LLVM’s middle end for agents [12]; and CodeRover-S repairs security bugs found by OSS-Fuzz [13]. Each makes an existing repair agent more selective. Our gate sits downstream, decides what reaches a human at all, and discards by default.

VI. ARTIFACT AND LIMITATIONS

We release the loop as CHIA nodes and tools, the framework’s own units of composition: a node is a function tagged with the worker resources it needs, a tool is what an agent is permitted to call, each with typed inputs and outputs, and both run inside a container image the release ships. Swapping the generator for the mutation arm is then one node substituted in the loop. The release includes a pull request against CHIA, the seed corpus with its map from commit to SDK release, and every generated input, oracle verdict, reduced case, repair transcript and gate decision.

What is not built. Nothing has run end to end; the stages are verified one at a time. The generator agent does not exist, and the claim that reading a CIRCT fix yields productive hypotheses rests on AFuzz’s results on JavaScript engines, not on this compiler. No bug has been found by this loop, and no CIRCT assertion has been seen firing on a generated input. The second oracle has run once, on a hand-written twelve-line design where both simulators agreed, and neither of its harness generators exists. The mutation arm, the assertions-on image builder, the deduplication rules and the budget file are specified and unwritten. The 187 seeds rest on a subject-keyword filter, and no diffs were read to confirm that each one is a bug fix. The cost of running with assertions on is unmeasured.

Scope. The loop does not repair wrong-output bugs, because the oracle that would find them is report-only. It does not replace maintainers. It converts the human event from writing a bug report into reading and approving one.

ACKNOWLEDGEMENTS

We thank the CHIA developers for their support, and the A³ hackathon organisers and sponsors.

AI-assistance disclosure. Claude (Anthropic) and Gemini (Google) were used for drafting, code generation and editorial

revision. The human authors made every claim, design decision and number their own, and are responsible for the entire contents, tone and quality of this paper.

REFERENCES

- [1] A. Cui, F. Hermida-Rivera, J. Toubes, R. Gupta, J. Fang, C. L. Zhang, E. Schwarz, J. Kim, Y. S. Shao, B. Nikolic, C. W. Fletcher, and S. Karandikar, “CHIA: An open-source framework for principled, agentic AI-driven hardware/software co-design research,” *arXiv preprint arXiv:2606.27350*, 2026.
- [2] B. Wang, P. Wang, C. Chen, M. Deng, J. Shi, Q. Sun, C. Yang, Y. Lin, Z. Yang, J. Chen, J. Sun, and D. Lo, “Mut4All: Fuzzing compilers via LLM-synthesized mutators learned from bug reports,” *arXiv preprint arXiv:2507.19275*, 2025.
- [3] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024.
- [4] X. Yang, Y. Chen, E. Eide, and J. Regehr, “Finding and understanding bugs in C compilers,” in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2011, pp. 283–294.
- [5] C. Suo, J. Chen, S. Liu, J. Jiang, Y. Zhao, and J. Wang, “Fuzzing MLIR compiler infrastructure via operation dependency analysis,” in *Proc. 33rd ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA)*, 2024, pp. 1287–1299.
- [6] B. Cao, W. Tong, Z. Tang, Z. Wang, H. Huang, and Y. Yan, “Nüwa: Enhancing MLIR fuzzing with LLM-driven generation and adaptive mutation,” in *Proc. IEEE Int. Conf. on Software Maintenance and Evolution (ICSME)*, 2025.
- [7] C. Suo, J. Wang, Y. Wang, J. Jiang, Q. Shen, and J. Chen, “DESIL: Detecting silent bugs in MLIR compiler infrastructure,” *arXiv preprint arXiv:2504.01379*, 2025.
- [8] Z. Sun, J. Liang, W. Wang, C. Suo, J. Chen, and F. Xu, “Interleaved learning and exploration: A self-adaptive fuzz testing framework for MLIR,” in *Proc. IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2025, arXiv:2510.07815.
- [9] J. Park and I. Yun, “Agentic fuzzing: Opportunities and challenges,” *arXiv preprint arXiv:2605.10074*, 2026.
- [10] A. Zeller and R. Hildebrandt, “Simplifying and isolating failure-inducing input,” *IEEE Trans. Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.
- [11] J. Cambroner, M. Tufano, S. Shi, R. Wei, G. Uy, R. Cheng, C.-J. Liu, S. Pan, S. Chandra, and P. Rondon, “Abstain and validate: A dual-LLM policy for reducing noise in agentic program repair,” in *Proc. IEEE/ACM Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2026, arXiv:2510.03217.
- [12] Y. Zheng, C. Li, S. Li, Y. Zhang, and Z. Su, “Agentic harness for real-world compilers,” *arXiv preprint arXiv:2603.20075*, 2026.
- [13] Y. Zhang, J. Wang, D. Berzin, M. Mirchev, and A. Roychoudhury, “Fixing security vulnerabilities with agentic AI in OSS-Fuzz,” in *Proc. IEEE/ACM Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2026, pp. 374–384.